コイン式ロック付きスマホスタンド 実証の手順書

Oct 14, 2026 · @Someone

この手順書の使い方

レベル0から6まで、1つずつ難しくなるように並べています。各レベルの最後にある「合格条件」にすべてチェックがついたら、次のレベルに進んでよい、というルールです（これがこの手順書の「許可証」です）。

各レベルは「目的（なぜやるか）」「やること」「合格条件」の3つでできています。うまくいかないときは、最後の「困ったとき」を見てください。

この手順書は「作るものは全部自分で理解している」ことを前提にしています。そのため合格条件は2種類あります。

- **動作チェック**：実際に動かして確かめること
- **理解チェック**：何も見ずに、自分の言葉で説明できること（説明できなければ、そこだけClaudeに質問する）

コードはコピーしても構いませんが、貼り付けたあとに「数字を1つ変えて、動きが予想どおりに変わるか」を必ず1回試してください。予想が当たれば、その行は理解できています。

レベル	場所	できるようになること
0 準備	PCと実物	Arduinoにプログラムを書き込める
1	Tinkercad	カバーを閉めるとタイマーが始まる
2	Tinkercad	決めた時間がたつと払い出し用サーボが動く
3	Tinkercad	コインのスイッチで解除用サーボが動く
4	Tinkercad	1〜3を1つのプログラムにまとめ、おかしな動きをなくす
5	実物	レベル4と同じことを、実物のブレッドボードで動かす
6	実物＋3Dプリント	3Dプリントの部品と組み合わせてデモをする

ピンの割り当て（全レベル共通）

Tinkercadでも実物でも、つなぐピンは同じにします。こうしておくと、Tinkercadで書いたプログラムをそのまま実物で使えます。

つなぐもの	Arduinoのピン	メモ
カバーのスイッチ	D2	閉めている間ずっと押される
コインのスイッチ	D3	コインが通る一瞬だけ押される
解除用サーボ（信号線）	D9	爪を引いてロックを外す
払い出し用サーボ（信号線）	D10	コインを1枚押し出す
スイッチのもう片方・サーボのGND・電池の−	GND	すべて同じ「−」の列にまとめる
サーボの電源（＋）	電池の＋	Arduinoの5Vにはつながない

レベル0　準備

目的：Tinkercadで試す前に、実物のArduinoにプログラムを書き込めることを確かめます。ここが通らないと、レベル5で「配線が悪いのか、書き込みが悪いのか」が区別できなくなります。

やること

1. tinkercad.comでアカウントを作り、「作成」→「回路」で空の回路を開けることを確かめる。
2. PCにArduino IDEを入れる。「ボードマネージャ」で「Arduino UNO R4 Boards」を追加する（R4 Minimaを使うために必要）。
3. UNO R4 MinimaをUSBケーブルでPCにつなぎ、ボードに「Arduino UNO R4 Minima」、ポートにそれらしいものを選ぶ。
4. 「ファイル」→「スケッチ例」→「01.Basics」→「Blink」を開き、書き込む。
5. 手元の部品を下の表で確かめる。

部品	数	状態	Tinkercadで代わりに使う部品
Arduino UNO R4 Minima	1	購入済み	Arduino Uno R3（R4はTinkercadにない。今回の使い方ならピンもコードも同じ）
サーボ FS90	2	購入済み	Micro Servo
マイクロスイッチ SS-10GL13（レバー付き）	2〜3	**未購入**（レベル5までに買う。秋月 114659）	カバー用はスライドスイッチ、コイン用は押しボタン
ブレッドボード、ジャンパーワイヤー	1セット	購入済み	ブレッドボード（小）
ワニ口クリップ付きコード	5本	購入済み	不要（普通の線でつなぐ）
電池ボックス 単3×4本＋単3アルカリ電池4本	1	購入済み	電池（単3×4本、6V）。見つからなければArduinoの5Vで代用
USBケーブル（Type-C、データ通信できるもの）	1	購入済み	不要
ノギス（あれば）	1	確認	不要（レベル6で部品の寸法を測るのに使う）

合格条件

動作チェック

- [ ] Blinkを書き込むと、ボードの「L」と書かれたLEDが1秒ごとに点滅する
- [ ] Blinkの `delay(1000)` を `delay(200)` に変えて書き込むと、点滅が速くなる
- [ ] Tinkercadで空の回路を作り、Arduino Uno R3を置ける

理解チェック

- [ ] `setup()` と `loop()` の違いを説明できる（setupは電源を入れたとき1回だけ、loopはそのあとずっと繰り返す）
- [ ] `delay(1000)` の1000の単位を言える（ミリ秒、1000ミリ秒＝1秒）

レベル1　カバーのスイッチでタイマーを始める（Tinkercad）

目的：「スイッチの状態を読む」「時間を測る」という、この作品の土台になる2つを覚えます。

やること

1. 新しい回路を作り、右の部品一覧を「基本」から「すべて」に切り替える。
2. 「Arduino Uno R3」「ブレッドボード（小）」「スライドスイッチ」を置く。
3. 配線する（端子をクリックして、もう一方の端子をクリックするとつながる）。
   - ArduinoのGND → ブレッドボードの「−」の列（青い線の列）
   - スライドスイッチの真ん中の端子 → ArduinoのD2
   - スライドスイッチの片側の端子 → 「−」の列
   - もう片側の端子は何にもつながない
4. 「コード」→「ブロック」を「文字」に切り替え、中身を全部消して下のコードを貼る。
5. 「シミュレーションを開始」を押し、コード画面の下の「シリアルモニタ」を開く。スライドスイッチをクリックして切り替える。

```
const int COVER_PIN = 2;        // カバーのスイッチをつないだピン

bool timerRunning = false;      // タイマーが動いているか（true/false）
unsigned long startTime = 0;    // タイマーを始めた時刻（ミリ秒）
unsigned long lastPrint = 0;    // 最後に画面に表示した時刻

void setup() {
  pinMode(COVER_PIN, INPUT_PULLUP);  // 内部の抵抗でHIGHにしておく
  Serial.begin(9600);                 // PCとの通信を始める（速さ9600）
  Serial.println("ready");
}

void loop() {
  // スイッチがGNDにつながっている（LOW）＝カバーが閉まっている
  bool coverClosed = (digitalRead(COVER_PIN) == LOW);

  // 閉まっていて、まだタイマーが動いていない → 今が「閉まった瞬間」
  if (coverClosed && !timerRunning) {
    timerRunning = true;
    startTime = millis();   // 今の時刻を覚えておく
    Serial.println("cover closed -> timer start");
  }

  // 開いていて、タイマーが動いている → 今が「開いた瞬間」
  if (!coverClosed && timerRunning) {
    timerRunning = false;
    Serial.println("cover opened -> timer stop");
  }

  // タイマーが動いている間は、1秒ごとに経過時間を表示
  if (timerRunning && millis() - lastPrint >= 1000) {
```

```
    lastPrint = millis();
    Serial.print("elapsed: ");
    Serial.print((millis() - startTime) / 1000);  // ミリ秒 → 秒
    Serial.println(" s");
  }
}
```

ここで覚える3つのこと

- **INPUT_PULLUP**：何もつながっていないピンは、HIGHかLOWかが決まらずフラフラします。`INPUT_PULLUP`にするとArduinoの中の抵抗がピンをHIGH（5V側）に引っ張っておくので、スイッチが入ってGNDにつながったときだけLOWになります。だから「押された＝LOW」と、ふつうの感覚と逆になります。外に抵抗をつけなくてよいのが利点です。
- **millis()**：Arduinoの電源が入ってから何ミリ秒たったかを返します。「今の時刻 − 始めた時刻」で経過時間がわかります。`delay()`と違って待たないので、時間を測りながらほかのこと（スイッチを読むなど）もできます。
- **「瞬間」の見つけ方**：`timerRunning`という旗（フラグ）を持っておき、「閉まっている」かつ「旗が下りている」ときだけ処理して旗を上げます。こうすると、閉まっている間ずっとではなく、閉まった瞬間に1回だけ反応します。

合格条件

動作チェック

- [ ] スイッチを切り替えると、シリアルモニタに`cover closed -> timer start`と出て、秒数が1つずつ増える
- [ ] 逆に切り替えると`cover opened -> timer stop`と出て、秒数の表示が止まる
- [ ] もう一度閉めると、秒数が0から数え直しになる
- [ ] 表示の間隔の`1000`を`500`に変えると、0.5秒ごとに表示される（予想してから試す）

理解チェック

- [ ] なぜ「押された」がHIGHではなくLOWなのかを説明できる
- [ ] `delay()`ではなく`millis()`で時間を測る理由を説明できる
- [ ] `timerRunning`がなかったら何が起きるかを説明できる（ヒント：閉めている間ずっと`startTime`が今の時刻で上書きされる）

レベル2　時間がたつと払い出し用サーボが動く（Tinkercad）

目的：サーボの動かし方と、「サーボの電源はArduinoと分けて、GNDだけ共通にする」という配線の決まりを覚えます。

やること

1. レベル1の回路を「複製」して、そこに足していく（レベル1はそのまま残しておく）。

2. 「Micro Servo」と電池（単3×4本）を置く。
3. 配線を足す。サーボの3本の端子は、マウスを乗せると名前が出る。
   - 電池の＋ → ブレッドボードの「＋」の列（赤い線の列）
   - 電池の− → 「−」の列（レベル1でArduinoのGNDをつないだ列）
   - サーボの電源（＋） → 「＋」の列
   - サーボのGND → 「−」の列
   - サーボの信号 → ArduinoのD10
4. コードを下のものに置き換えて、シミュレーションを始める。

```
#include <Servo.h>              // サーボを動かすための部品集（ライブラリ）

const int COVER_PIN = 2;        // カバーのスイッチ
const int DISPENSE_PIN = 10;    // 払い出し用サーボの信号線

// コインを出す間隔。試すときは10秒、本番は 3600000UL（1時間）
const unsigned long COIN_INTERVAL = 10000UL;

// サーボの角度（0〜180度）。実物に合わせてレベル5で調整する
const int DISPENSE_HOME = 0;    // 待っているときの角度
const int DISPENSE_PUSH = 90;   // コインを押し出すときの角度

Servo dispenseServo;            // 払い出し用サーボを表す変数
bool timerRunning = false;
unsigned long startTime = 0;

// コインを1枚出す：押し出して、元に戻す
void dispenseCoin() {
  dispenseServo.write(DISPENSE_PUSH);
  delay(500);                   // サーボが回りきるのを待つ
  dispenseServo.write(DISPENSE_HOME);
  delay(500);
}

void setup() {
  pinMode(COVER_PIN, INPUT_PULLUP);
  dispenseServo.attach(DISPENSE_PIN);   // D10にサーボがつながっていると教える
  dispenseServo.write(DISPENSE_HOME);   // 最初は待ちの角度にしておく
  Serial.begin(9600);
  Serial.println("ready");
}

void loop() {
  bool coverClosed = (digitalRead(COVER_PIN) == LOW);
```

```
  if (coverClosed && !timerRunning) {
    timerRunning = true;
    startTime = millis();
    Serial.println("cover closed -> timer start");
  }
  if (!coverClosed && timerRunning) {
    timerRunning = false;
    Serial.println("cover opened -> timer stop");
  }

  // 決めた時間がたったら、コインを1枚出して、また0から数える
  if (timerRunning && millis() - startTime >= COIN_INTERVAL) {
    Serial.println("time up -> dispense 1 coin");
    dispenseCoin();
    startTime = millis();
  }
}
```

ここで覚える3つのこと

- **サーボの動かし方**：`write(角度)`で「その角度まで回って止まれ」と命令します。どれだけ回すかではなく、どの角度にいるかを指定するのがポイントです。
- **電源を分ける理由**：サーボは動き出す瞬間に大きな電流を使います。Arduinoの5Vからとると電圧が下がり、Arduinoが勝手にリセットされることがあります。だからサーボの電源は電池からとります。
- **GNDを共通にする理由**：信号線の「HIGH／LOW」は、GNDを0Vとしたときの電圧で決まります。ArduinoとサーボのGNDがつながっていないと、基準の0Vがずれて、サーボが信号を正しく読めません。

合格条件

動作チェック

- [ ] スイッチを閉めて10秒たつと`time up -> dispense 1 coin`と出て、サーボの腕が動いて戻る
- [ ] 閉めたままにすると、10秒ごとに繰り返す
- [ ] 10秒たつ前に開けると、サーボは動かない
- [ ] `DISPENSE_PUSH`を`45`にすると、腕の動きが小さくなる（予想してから試す）

理解チェック

- [ ] サーボの3本の線が、それぞれどこにつながっていて、何のためにあるか説明できる
- [ ] 電池の−とArduinoのGNDをつなぐ理由を説明できる
- [ ] 本番で1時間にするには、どの行をどう変えればよいか言える

レベル3　コインのスイッチで解除用サーボが動く（Tinkercad）

目的：「押された瞬間」を見つける方法と、スイッチのチャタリング（接点のバタつき）への対策を覚えます。

やること

1. レベル2の回路を「複製」して足していく。
2. 「押しボタン」と、2つ目の「Micro Servo」を置く。
3. 配線を足す。
   - 押しボタンをブレッドボードの真ん中の溝をまたぐように置く
   - 押しボタンの足の1本 → ArduinoのD3
   - その足と**対角にある足** → 「−」の列（足が4本あって迷ったら、対角の2本を使えば必ず動く）
   - 2つ目のサーボ：電源（＋）→「＋」の列、GND→「−」の列、信号→ArduinoのD9
4. コードを下のものに置き換えて、シミュレーションを始める。

```
#include <Servo.h>

const int COVER_PIN = 2;         // カバーのスイッチ
const int COIN_PIN = 3;          // コインのスイッチ
const int UNLOCK_PIN = 9;        // 解除用サーボの信号線
const int DISPENSE_PIN = 10;     // 払い出し用サーボの信号線

const unsigned long COIN_INTERVAL = 10000UL;   // 本番は 3600000UL

const int DISPENSE_HOME = 0;
const int DISPENSE_PUSH = 90;
const int UNLOCK_HOME = 0;       // 爪がかかったままの角度
const int UNLOCK_PULL = 90;      // 爪を引いてロックを外す角度

Servo dispenseServo;
Servo unlockServo;
bool timerRunning = false;
unsigned long startTime = 0;
bool lastCoin = false;           // 前回見たとき、コインのスイッチが押されていたか

void dispenseCoin() {
  dispenseServo.write(DISPENSE_PUSH);
  delay(500);
  dispenseServo.write(DISPENSE_HOME);
  delay(500);
}
```

```
// 爪を引いてロックを外し、2秒後に戻す
void unlock() {
  unlockServo.write(UNLOCK_PULL);
  delay(2000);                    // この間にカバーを開ける
  unlockServo.write(UNLOCK_HOME);
}

void setup() {
  pinMode(COVER_PIN, INPUT_PULLUP);
  pinMode(COIN_PIN, INPUT_PULLUP);
  dispenseServo.attach(DISPENSE_PIN);
  unlockServo.attach(UNLOCK_PIN);
  dispenseServo.write(DISPENSE_HOME);
  unlockServo.write(UNLOCK_HOME);
  Serial.begin(9600);
  Serial.println("ready");
}

void loop() {
  bool coverClosed = (digitalRead(COVER_PIN) == LOW);
  bool coinNow = (digitalRead(COIN_PIN) == LOW);

  if (coverClosed && !timerRunning) {
    timerRunning = true;
    startTime = millis();
    Serial.println("cover closed -> timer start");
  }
  if (!coverClosed && timerRunning) {
    timerRunning = false;
    Serial.println("cover opened -> timer stop");
  }

  // 前回は押されていなくて、今は押されている → 「押された瞬間」
  if (coinNow && !lastCoin) {
    delay(50);                    // チャタリングが収まるのを待つ
    if (coverClosed) {
      Serial.println("coin in -> unlock");
      timerRunning = false;       // タイマーを0に戻す
      unlock();
    }
  }
  lastCoin = coinNow;             // 次に比べるために、今の状態を覚えておく

  if (timerRunning && millis() - startTime >= COIN_INTERVAL) {
    Serial.println("time up -> dispense 1 coin");
```

```
    dispenseCoin();
    startTime = millis();
  }
}
```

ここで覚える3つのこと

- **押された瞬間の見つけ方（その2）**：レベル1は「旗」を使いましたが、ここでは「前回の状態」と「今の状態」を比べています。前回は押されていない、今は押されている、のときが押された瞬間です。コインは一瞬しかスイッチに触れないので、この見つけ方が合っています。
- **チャタリング**：機械式のスイッチは、押した瞬間に接点が細かく跳ねて、ON/OFFを何度か繰り返します。そのまま読むと「1枚入れたのに2回反応した」となるので、反応したら50ミリ秒待ってやり過ごします。Tinkercadではほとんど起きませんが、実物では起きます。
- **わざと残してある問題**：このコードには、レベル4で直す問題が2つあります。下の動作チェックで確かめてください。

合格条件

動作チェック

- [ ] カバーを閉めた状態でボタンを押すと、`coin in -> unlock`と出て、解除用サーボが動いて2秒後に戻る
- [ ] そのあと秒数が0から数え直しになる
- [ ] カバーを開けた状態でボタンを押しても、何も起きない
- [ ] 問題1を見つける：ボタンを押してから2秒以内にカバー（スライドスイッチ）を開けないと、どうなるか
- [ ] 問題2を見つける：サーボがコインを出している最中（約1秒）にボタンを押すと、どうなるか

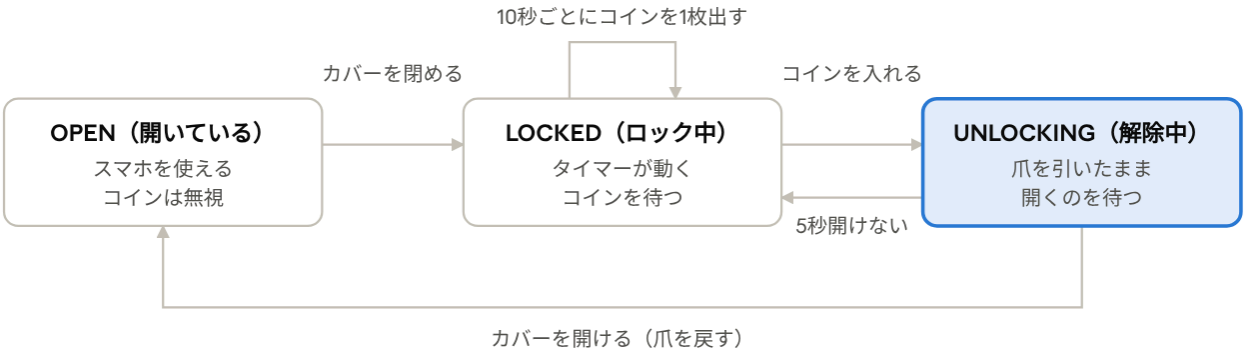
理解チェック

- [ ] `lastCoin = coinNow;`の行がなかったら何が起きるか説明できる
- [ ] チャタリングとは何か、なぜ`delay(50)`で防げるのか説明できる
- [ ] 問題1と問題2が、なぜ起きるのかを説明できる（ヒント：どちらも`delay()`が関係している）

レベル4　3つをまとめる（Tinkercad）

目的：レベル3で見つけた2つの問題を、「状態」という考え方で直します。今どの状態にいるかを1つの変数で持ち、状態ごとにやることを分けると、動きが図のとおりになり、説明もしやすくなります。

レベル4では「解除中」を足し、カバーが開くまで爪を引いたまま待つ



状態の流れ·3つの状態と5つの移り方

矢印が「何が起きたら、どの状態に移るか」です。レベル3の問題1は「解除中」がなかったせいで、問題2は`delay()`で待っている間スイッチを読めなかったせいで起きていました。

やること

1. 上の図を、何も見ずに紙に描けるようにする（コードより先に）。
2. レベル3の回路をそのまま使い、コードを下のものに置き換える。
3. 合格条件の動作チェックを、上から順に全部試す。

```
#include <Servo.h>

const int COVER_PIN = 2;
const int COIN_PIN = 3;
const int UNLOCK_PIN = 9;
const int DISPENSE_PIN = 10;

const unsigned long COIN_INTERVAL = 10000UL;  // 本番は 3600000UL（1時間）
const unsigned long OPEN_WAIT = 5000UL;       // 爪を引いてから、カバーが開くのを
待つ時間
const unsigned long PUSH_TIME = 500UL;        // 払い出し用サーボが押し出している
時間
const unsigned long DEBOUNCE = 200UL;         // この時間内の2回目の反応は無視する

const int DISPENSE_HOME = 0;
const int DISPENSE_PUSH = 90;
const int UNLOCK_HOME = 0;
const int UNLOCK_PULL = 90;

// 状態は3つ：OPEN（開いている）、LOCKED（ロック中）、UNLOCKING（解除中）
enum State { OPEN, LOCKED, UNLOCKING };
State state = OPEN;
```

```
Servo dispenseServo;
Servo unlockServo;

unsigned long startTime = 0;      // タイマーを始めた時刻
unsigned long unlockStart = 0;    // 爪を引き始めた時刻
unsigned long pushStart = 0;      // 押し出しを始めた時刻
bool pushing = false;             // 今、押し出している最中か
bool lastCoin = false;            // 前回、コインのスイッチが押されていたか
unsigned long lastCoinTime = 0;   // 最後にコインに反応した時刻

void setup() {
  pinMode(COVER_PIN, INPUT_PULLUP);
  pinMode(COIN_PIN, INPUT_PULLUP);
  dispenseServo.attach(DISPENSE_PIN);
  unlockServo.attach(UNLOCK_PIN);
  dispenseServo.write(DISPENSE_HOME);
  unlockServo.write(UNLOCK_HOME);
  Serial.begin(9600);
  Serial.println("ready");
}

void loop() {
  unsigned long now = millis();
  bool coverClosed = (digitalRead(COVER_PIN) == LOW);
  bool coinNow = (digitalRead(COIN_PIN) == LOW);

  // コインが入った瞬間（前の反応からDEBOUNCE以上たっているときだけ）
  bool coinIn = false;
  if (coinNow && !lastCoin && now - lastCoinTime >= DEBOUNCE) {
    coinIn = true;
    lastCoinTime = now;
  }
  lastCoin = coinNow;

  // 押し出してからPUSH_TIMEたったら戻す（delayで待たない）
  if (pushing && now - pushStart >= PUSH_TIME) {
    dispenseServo.write(DISPENSE_HOME);
    pushing = false;
  }

  switch (state) {
    case OPEN:        // カバーが開いている：閉められるのを待つ
      if (coinIn) {
        Serial.println("coin ignored (not locked)");
      }
```

```
      if (coverClosed) {
        state = LOCKED;
        startTime = now;
        Serial.println("OPEN -> LOCKED (timer start)");
      }
      break;

    case LOCKED:      // ロック中：時間を数え、コインを待つ
      if (now - startTime >= COIN_INTERVAL && !pushing) {
        dispenseServo.write(DISPENSE_PUSH);
        pushStart = now;
        pushing = true;
        startTime = now;
        Serial.println("time up -> dispense 1 coin");
      }
      if (coinIn) {
        unlockServo.write(UNLOCK_PULL);
        unlockStart = now;
        state = UNLOCKING;
        Serial.println("coin in -> UNLOCKING");
      }
      if (!coverClosed && state == LOCKED) {   // 本来は起きない
        state = OPEN;
        Serial.println("cover opened while locked?");
      }
      break;

    case UNLOCKING:   // 爪を引いたまま、カバーが開くのを待つ
      if (!coverClosed) {
        unlockServo.write(UNLOCK_HOME);
        state = OPEN;
        Serial.println("cover opened -> OPEN");
      } else if (now - unlockStart >= OPEN_WAIT) {
        unlockServo.write(UNLOCK_HOME);
        state = LOCKED;
        startTime = now;
        Serial.println("not opened -> back to LOCKED");
      }
      break;
  }
}
```

ここで覚える3つのこと

- **状態（ステートマシン）**：`state`という変数に「今どの状態か」を入れ、`switch`で状態ごとの処理に分けます。図の箱が`case`、矢印が`state = ...`の行にそのまま対応しています。
- **delayを使わずに待つ**：「始めた時刻を覚えておき、毎回の`loop()`で、決めた時間がたったかを確かめる」書き方です。待っている間も`loop()`は回り続けるので、スイッチを読み逃しません。払い出し、解除の待ち、チャタリング対策の3か所で同じ形を使っています。
- **決めごとを数字にする**：5秒待っても開けなかったときは、爪を戻してロック中に戻ります（入れたコインは戻りません）。「開けるまでずっと待つ」にしたいなら、`else if`の部分を消せば変えられます。どちらにするかは自分で決めてください。

合格条件

動作チェック

- [ ] 閉める → 10秒たつ → 払い出し用サーボが動く
- [ ] 閉める → ボタンを押す → 5秒以内にスライドスイッチで開ける → `cover opened -> OPEN`と出る（レベル3の問題1が直っている）
- [ ] 閉める → ボタンを押す → 開けずに5秒待つ → `not opened -> back to LOCKED`と出て、秒数が0から始まる
- [ ] サーボがコインを出している最中にボタンを押しても、ちゃんと反応する（レベル3の問題2が直っている）
- [ ] 開いているときにボタンを押すと、`coin ignored`と出るだけで何も動かない

理解チェック

- [ ] 状態の図を何も見ずに描き、それぞれの矢印がコードのどの行かを指させる
- [ ] レベル3の問題1と問題2が、このコードではなぜ起きないのかを説明できる
- [ ] `PUSH_TIME`を`2000`にすると何が起きるか、予想してから試せる

レベル5　実物に移す

目的：Tinkercadで動いたものを、実物のブレッドボードで動かします。サーボの力、電池の減り、スイッチの押し心地、角度のずれなど、Tinkercadでは見えない問題をここで見つけます。

やること

1. マイクロスイッチ（SS-10GL13、レバー付き）を買う。カバー用、コイン用、予備で3個。
2. 実物の配線をする。Tinkercadとつなぐピンは同じ。電池ボックスのスイッチはOFFにしたまま配線する。
    - ArduinoはUSBでPCにつなぎ、そこから電気をもらう

- 電池ボックスの赤い線（＋）→「＋」の列、黒い線（−）→「−」の列、「−」の列→ArduinoのGND
- FS90の線は、茶＝GND、赤＝＋、オレンジ＝信号が一般的（買ったものの色を確認する）。コネクタの穴にジャンパーワイヤーを差してブレッドボードへ
- マイクロスイッチには端子が3つある（COM、NO、NC。本体に刻印がある）。**COM→「−」の列、NO→D2またはD3**。NCは使わない
- マイクロスイッチの端子はワニ口クリップで挟む。クリップ同士が触れないようにする

3. 1つずつ足しながら確かめる（一度に全部つながない）。
   a. カバー用マイクロスイッチだけつなぎ、レベル1のコードを書き込む
   b. 払い出し用サーボを足し、レベル2のコードを書き込む
   c. コイン用マイクロスイッチと解除用サーボを足し、レベル4のコードを書き込む
4. サーボの角度を決める。サーボの腕（ホーン）は、コードで`HOME`の角度にしてから取り付ける。こうすると「待っている位置」がずれない。そのあと`PUSH`や`PULL`の数字を少しずつ変えて、ちょうどよい角度を探す。

守ること

- 電池の＋を、Arduinoの5Vや信号のピンにつながない
- 配線を変えるときは、電池ボックスのスイッチをOFFにし、USBも抜く
- サーボが端にぶつかってジーッと鳴り続けたら、すぐ電池をOFFにする（角度が行きすぎている）

合格条件

動作チェック

- [ ] レベル4の動作チェックが、実物でも全部通る
- [ ] コイン用マイクロスイッチを指で10回押して、10回とも1回ずつだけ反応する（2回反応しない）
- [ ] サーボが動くときに、Arduinoがリセットされない（`ready`が途中でまた出ない）
- [ ] 止まっているサーボが、小刻みに震え続けない

理解チェック

- [ ] 何も見ずに、実物の配線図を紙に描ける
- [ ] マイクロスイッチのCOM、NO、NCの違いを説明できる
- [ ] NOではなくNCにつないだら動きがどう変わるか、予想を言える（そのあと実際に試す）

レベル6　3Dプリントの部品と組み合わせてデモをする

目的：本体を全部作る前に、仕組みごとに小さな「試験台」を3Dプリントして、本当に動くか確かめます。ここで決まった寸法（コインの大きさ、すきま、角度）が、そのまま本体の設計に使えます。必要な部品は次の「デモ用に欲しい3Dプリント部品」にまとめています。

やること（上から順に。1つ通ってから次へ）

1. **3Dのソフトを決める**：設計ソフトはまだ決まっていないので、最初に1つ選ぶ。部品はどれも箱・円柱・板の組み合わせでできる形なので、どのソフトでも作れる。
2. **すきまを確かめる**：3Dプリントの穴は、設計より少し小さくできあがる。直径10 mmの棒と、10.0／10.2／10.4／10.6 mmの穴が開いた板を印刷し、どのすきまでちょうどよくはまるか確かめる。以後の設計はこの数字を使う。
3. **テスト用コイン**を5枚ほど印刷し、厚みと直径を決める。
4. **コイン投入口とスロープ**を印刷してコイン用マイクロスイッチを取り付け、コインを落としてスイッチが入るか確かめる。
5. **コインの筒と押し出し板**を印刷して払い出し用サーボを取り付け、1回に1枚だけ出るか確かめる。
6. **ロックの試験台（扉＋爪）**を印刷し、扉を手で閉めると爪がかかり、解除用サーボで外れるか確かめる。カバー用マイクロスイッチもここにつける。バネの代わりに輪ゴムでもよい。
7. **ベース板**に全部を並べて固定し、デモの流れを通しで行う。

デモの流れ（発表で見せる順番）

1. スマホを置いて扉を手で閉める → 爪がかかってロックされ、タイマーが始まる
2. 10秒たつ → コインが1枚出てくる
3. そのコインを投入口に入れる → 爪が外れ、扉が開けられる
4. 扉を開けてスマホを取り出す → もう一度閉めると、タイマーが0から始まる

PCの画面にシリアルモニタを出しておくと、裏で何が起きているかも見せられます。

合格条件

動作チェック

- [ ] テスト用コインを投入口から10回落として、10回ともスイッチが入る
- [ ] 払い出しを10回続けて、10回とも1枚だけ出る（0枚や2枚にならない）
- [ ] 扉を手で閉めると爪がかかり、手で引っぱっても開かない
- [ ] コインを入れると爪が外れて扉が開く。これを10回続けて失敗しない
- [ ] デモの流れ1〜4を、止まらずに3回続けてできる

理解チェック

- [ ] 3Dプリントの穴に「すきま」を足す理由と、自分のプリンタで使う数字を言える

- [ ] コインが軽すぎる、薄すぎるとどんな失敗が起きるか説明できる
- [ ] 爪がかかる向きと外れる向き、サーボがどちらに引けばよいかを図で描ける

デモ用に欲しい3Dプリント部品

デモに最低限必要なのは「必須」の5つです。寸法は、まず実物（サーボ、マイクロスイッチ、スマホ）をノギスで測ってから決めます。数字を予想で決めないのが大事です。

部品	優先度	何を確かめるか	最低限の形	先に測るもの
すきまの試し板	必須	穴を何mm大きくすればちょうどはまるか	直径10 mmの棒＋大きさの違う穴4つを開けた板	なし
テスト用コイン（5枚ほど）	必須	マイクロスイッチを押せる重さか、筒の中で引っかからないか	円盤。厚みを変えて数種類	マイクロスイッチのレバーの長さと引っ込む幅
コイン投入口＋スロープ（スイッチ取り付け台つき）	必須	落ちたコインが必ずレバーを押すか	斜めの溝＋スイッチをねじで止める台	マイクロスイッチの取り付け穴の間隔と大きさ
コインの筒＋押し出し板（サーボ取り付け台つき）	必須	1回に1枚だけ出るか	コインを積む縦の筒。下に1枚分の隙間。サーボの腕で一番下の1枚を押し出す	FS90の本体の大きさ、取り付け穴、腕（ホーン）の長さ
ロックの試験台（扉＋爪）	必須	手で閉めるとかかり、サーボで外れるか	蝶番いで開く小さな扉＋輪ゴムで戻る爪。爪とサーボは糸か鉄線でつなぐ	FS90の大きさ、カバー用マイクロスイッチの取り付け穴
ベース板	あるとよい	デモ中に部品がずれない	上の試験台とArduino、ブレッドボード、電池を並べて止める板	Arduinoとブレッドボードの大きさ
スマホを立てる台	あるとよい	スマホを置くという動きを見せられる	斜めの板＋下の受け	使うiPhoneの大きさ

まだ決めていないこと：入れたコインをそのまま筒に戻す「循環型」にするか。デモでは、投入口と筒を別々にして、コインは手で筒に戻す形で十分です。

困ったとき

上から順に確かめてください。質問するときは「レベルの番号」「配線の写真かスクリーンショット」「シリアルモニタに出ている文字」の3つがあると、原因をすぐ絞れます。

症状	よくある原因	確かめること
サーボがまったく動かない	信号線のピン違い、電池のスイッチがOFF、GNDがつながっていない	信号線がD9／D10か。電池の−とArduinoのGNDが同じ列か
サーボが小刻みに震える、Arduinoが勝手にリセットされる	電池が弱い、サーボの電源をArduinoの5Vからとっている	新しい電池にする。サーボの＋が電池につながっているか
スイッチの反応が逆（押していないのに押したことになる）	マイクロスイッチをNCにつないでいる	NOにつなぎ直す
1回押しただけで2回反応する	チャタリング	待つ時間（ミリ秒）を長くする
何も押していないのに反応がバラバラ	`INPUT_PULLUP`を書き忘れている	`setup()`の`pinMode`を見る
書き込みができない	ボードかポートの選び間違い、充電専用のUSBケーブル	ボードが「Arduino UNO R4 Minima」か。別のケーブルで試す
シリアルモニタが文字化けする	通信の速さが合っていない	シリアルモニタ右下の速さを9600にする
Tinkercadで時間が遅く進む	シミュレーションが実際の時間より遅い	気にしなくてよい。画面上部のシミュレーション時間を見る